
🔴 **SAVOL 46:**

`[PYQT5]` IKKI SON KIRITIB ULARNING YIG'INDISI, AYIRMASI, KO'PAYTMASI VA BO'LINMASINI KO'RSATADIGAN ODDIY KALKULYATOR DASTURINI YOZING. `QLINEEDIT`, `QPUSHBUTTON` VA `QLABEL` DAN FOYDALANING.

```
import sys
from PyQt5.QtWidgets import (
    QApplication, QMainWindow, QWidget,
    QVBoxLayout, QHBoxLayout, QGridLayout,
    QLabel, QLineEdit, QPushButton, QMessageBox
)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont


class Kalkulyator(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Kalkulyator")
        self.setFixedSize(420, 520)
        self.setStyleSheet("background-color: #1e272e;")
        self.interfeys_yaratish()

    def interfeys_yaratish(self):
        markaziy = QWidget()
        self.setCentralWidget(markaziy)
        asosiy = QVBoxLayout(markaziy)
        asosiy.setContentsMargins(25, 25, 25, 25)
        asosiy.setSpacing(15)

        # ── Sarlavha ──────────────────────────────
        sarlavha = QLabel("🧮 Kalkulyator")
        sarlavha.setAlignment(Qt.AlignCenter)
        sarlavha.setStyleSheet("""
            font-size: 22px;
            font-weight: bold;
            color: white;
            background-color: #2c3e50;
            border-radius: 10px;
            padding: 12px;
        """)
        asosiy.addWidget(sarlavha)

        # ── Birinchi son ──────────────────────────
        label1 = QLabel("Birinchi son:")
        label1.setStyleSheet("color: #bdc3c7; font-size: 14px;")
        asosiy.addWidget(label1)

        self.son1 = QLineEdit()
        self.son1.setPlaceholderText("Masalan: 15.5")
        self.son1.setStyleSheet(self.input_stili())
        self.son1.setMinimumHeight(45)
```

```
asosiy.addWidget(self.son1)

# ── Ikkinchi son ──────────────────────────
label2 = QLabel("Ikkinchi son:")
label2.setStyleSheet("color: #bdc3c7; font-size: 14px;")
asosiy.addWidget(label2)

self.son2 = QLineEdit()
self.son2.setPlaceholderText("Masalan: 4")
self.son2.setStyleSheet(self.input_stili())
self.son2.setMinimumHeight(45)
asosiy.addWidget(self.son2)

# ── Amal tugmalari ────────────────────────
tugmalar_label = QLabel("Amal tanlang:")
tugmalar_label.setStyleSheet(
    "color: #bdc3c7; font-size: 14px;"
)
asosiy.addWidget(tugmalar_label)

grid = QGridLayout()
grid.setSpacing(10)

tugmalar = [
    ("➕  Qo'shish",   "#27ae60", "#1e8449", self.qoshish),
    ("➖  Ayirish",    "#2980b9", "#1a6fa8", self.ayirish),
    ("✖️  Ko'paytirish","#8e44ad", "#7d3c98", self.kopaytirish),
    ("➗  Bo'lish",    "#e67e22", "#ca6f1e", self.bolish),
]

for i, (matn, rang, pressed_rang, funksiya) in enumerate(tugmalar):
    tugma = QPushButton(matn)
    tugma.setMinimumHeight(48)
    tugma.setStyleSheet(f"""
        QPushButton {{
            background-color: {rang};
            color: white;
            font-size: 15px;
            font-weight: bold;
            border-radius: 10px;
            border: none;
        }}
        QPushButton:hover {{
            background-color: {pressed_rang};
        }}
        QPushButton:pressed {{
            background-color: {pressed_rang};
            padding-top: 3px;
        }}
    """)
    tugma.clicked.connect(funksiya)
    grid.addWidget(tugma, i // 2, i % 2)

asosiy.addLayout(grid)

# ── Natija ────────────────────────────────
natija_label = QLabel("Natija:")
```

```
        natija_label.setStyleSheet(
            "color: #bdc3c7; font-size: 14px;"
        )
        asosiy.addWidget(natija_label)

        self.natija = QLabel("—")
        self.natija.setAlignment(Qt.AlignCenter)
        self.natija.setMinimumHeight(65)
        self.natija.setStyleSheet("""
            QLabel {
                background-color: #2c3e50;
                color: #2ecc71;
                font-size: 26px;
                font-weight: bold;
                border-radius: 12px;
                border: 2px solid #3d5a6e;
                padding: 10px;
            }
        """)
        asosiy.addWidget(self.natija)

        # ── Tozalash tugmasi ──────────────────────
        tozala = QPushButton("🗑  Tozalash")
        tozala.setMinimumHeight(45)
        tozala.setStyleSheet("""
            QPushButton {
                background-color: #7f8c8d;
                color: white;
                font-size: 14px;
                font-weight: bold;
                border-radius: 10px;
                border: none;
            }
            QPushButton:hover { background-color: #95a5a6; }
            QPushButton:pressed { background-color: #6c7a7d; }
        """)
        tozala.clicked.connect(self.tozalash)
        asosiy.addWidget(tozala)

    # ── Yordamchi funksiyalar ─────────────────────

    def input_stili(self):
        return """
            QLineEdit {
                background-color: #2c3e50;
                color: white;
                font-size: 16px;
                border: 2px solid #3d5a6e;
                border-radius: 10px;
                padding: 8px 14px;
            }
            QLineEdit:focus {
                border: 2px solid #3498db;
                background-color: #34495e;
            }
        """
```

```
def sonlarni_ol(self):
    """
    Inputlardan sonlarni oladi.
    Xato bo'lsa None qaytaradi.
    """
    try:
        a = float(self.son1.text().strip())
        b = float(self.son2.text().strip())
        return a, b
    except ValueError:
        QMessageBox.warning(
            self,
            "Xato",
            "Iltimos, ikkala maydonga ham\n"
            "to'g'ri son kiriting!"
        )
        return None, None

def natijani_korsot(self, amal, a, b, javob):
    """Natijani chiroyli formatda ko'rsatadi."""
    if isinstance(javob, float):
        # Butun son bo'lsa .0 ni ko'rsatmaslik
        if javob == int(javob):
            javob_str = str(int(javob))
        else:
            javob_str = f"{javob:.4f}".rstrip("0")
    else:
        javob_str = str(javob)

    self.natija.setText(
        f"{a} {amal} {b} = {javob_str}"
    )
    self.natija.setStyleSheet("""
        QLabel {
            background-color: #1a3a2a;
            color: #2ecc71;
            font-size: 22px;
            font-weight: bold;
            border-radius: 12px;
            border: 2px solid #27ae60;
            padding: 10px;
        }
    """)

# ── Amal funksiyalari ──────────────────────────

def qoshish(self):
    a, b = self.sonlarni_ol()
    if a is not None:
        self.natijani_korsot("+", a, b, a + b)

def ayirish(self):
    a, b = self.sonlarni_ol()
    if a is not None:
        self.natijani_korsot("-", a, b, a - b)

def kopaytirish(self):
```

```
        a, b = self.sonlarni_ol()
        if a is not None:
            self.natijani_korsot("×", a, b, a * b)

    def bolish(self):
        a, b = self.sonlarni_ol()
        if a is not None:
            if b == 0:
                QMessageBox.critical(
                    self,
                    "Matematik Xato",
                    "Nolga bo'lish mumkin emas!"
                )
                self.natija.setText("⚠️  Nolga bo'lib bo'lmaydi")
                self.natija.setStyleSheet("""
                    QLabel {
                        background-color: #3d1515;
                        color: #e74c3c;
                        font-size: 18px;
                        font-weight: bold;
                        border-radius: 12px;
                        border: 2px solid #e74c3c;
                        padding: 10px;
                    }
                """)
                return
            self.natijani_korsot("÷", a, b, a / b)

    def tozalash(self):
        self.son1.clear()
        self.son2.clear()
        self.natija.setText("—")
        self.natija.setStyleSheet("""
            QLabel {
                background-color: #2c3e50;
                color: #2ecc71;
                font-size: 26px;
                font-weight: bold;
                border-radius: 12px;
                border: 2px solid #3d5a6e;
                padding: 10px;
            }
        """)
        self.son1.setFocus()


if __name__ == "__main__":
    app = QApplication(sys.argv)
    oyna = Kalkulyator()
    oyna.show()
    sys.exit(app.exec_())
```

Dastur Tuzilishi

```
Kalkulyator (QMainWindow)
├── Sarlavha (QLabel)
├── Birinchi son (QLineEdit)
├── Ikkinchi son (QLineEdit)
├── Tugmalar (4 ta QPushButton — Grid)
│     ├── ➕ Qo'shish
│     ├── ➖ Ayirish
│     ├── ✖️ Ko'paytirish
│     └── ➗ Bo'lish
├── Natija (QLabel)
└── Tozalash (QPushButton)
```

Asosiy Texnik Tushunchalar

```
sonlarni_ol():
  → float() → Har ikki inputni son ga aylantirish
  → try/except → Noto'g'ri kiritish uchun xato ushlab olish

natijani_korsot():
  → Butun son bo'lsa → ".0" ko'rsatmaslik
  → Kasr bo'lsa → Ortiqcha nollarni olib tashlash
  → rstrip("0") → "3.5000" → "3.5"

bolish():
  → b == 0 tekshiruvi → Nolga bo'lish oldini olish
  → QMessageBox.critical → Xato xabari
  → Natija label rangi qizilga o'zgaradi

Signal-Slot:
  tugma.clicked.connect(self.qoshish) → Tugma bosilsa funksiya
```

> 💡 **Eslab qol:** `float()` konversiyasi `try/except` ichida bo'lishi **shart** — foydalanuvchi "abc" yoki bo'sh maydon kiritsa dastur ishdan chiqmasin. Nolga bo'lishni ham alohida tekshirish kerak — bu **dasturchi mas'uliyati!**

🔴 **SAVOL 47:**

`[SOCKET]` PYTHON DA ODDIY `ECHO-SERVER` YARATING: SERVER KLIENTDAN KELGAN HAR QANDAY XABARNI KATTA HARFLARGA O'GIRIB QAYTARSIN. SERVER VA KLIENT QISMLARINI ALOHIDA FAYLLARDA YOZING.

---

server.py

```python
# server.py
import socket
import threading
import datetime


HOST = "0.0.0.0"   # Barcha interfeyslarda tinglash
```

```python
PORT = 12345


def vaqt():
    return datetime.datetime.now().strftime("%H:%M:%S")


def klient_handler(klient_sock, manzil):
    """
    Har bir klient uchun alohida thread da ishlaydi.
    Kelgan xabarni KATTA HARFGA o'girib qaytaradi.
    """
    print(f"[{vaqt()}] ✅ Yangi ulanish: {manzil[0]}:{manzil[1]}")

    try:
        while True:
            # 1. Xabar qabul qilish
            data = klient_sock.recv(1024)

            # 2. Bo'sh data → Klient uzildi
            if not data:
                print(f"[{vaqt()}] 🔌 Klient uzildi: {manzil[0]}:{manzil[1]}")
                break

            # 3. Dekod qilish
            xabar = data.decode("utf-8").strip()
            print(f"[{vaqt()}] 📨 Keldi [{manzil[0]}]: '{xabar}'")

            # 4. Maxsus buyruqlar tekshirish
            if xabar.lower() == "exit":
                klient_sock.send("Xayr! Ulanish yopildi.".encode("utf-8"))
                print(f"[{vaqt()}] 👋 Klient chiqdi: {manzil[0]}")
                break

            if xabar.lower() == "ping":
                javob = "PONG"
            else:
                # 5. Asosiy echo: KATTA HARFGA o'girish
                javob = xabar.upper()

            # 6. Javob yuborish
            klient_sock.send(javob.encode("utf-8"))
            print(f"[{vaqt()}] 📤 Yuborildi [{manzil[0]}]: '{javob}'")

    except ConnectionResetError:
        print(f"[{vaqt()}] ⚠️  Klient to'satdan uzildi: {manzil[0]}")
    except Exception as e:
        print(f"[{vaqt()}] ❌ Xato [{manzil[0]}]: {e}")
    finally:
        klient_sock.close()
        print(f"[{vaqt()}] 🔒 Socket yopildi: {manzil[0]}")


def serverni_ishga_tushir():
    # Socket yaratish
    server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
    # SO_REUSEADDR → Dastur qayta ishga tushganda port darhol bo'shaydi
    server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    # Manzil va port biriktirish
    server_sock.bind((HOST, PORT))

    # Tinglash rejimi (max 5 ta navbatda kutuvchi)
    server_sock.listen(5)

    print("=" * 45)
    print("       🚀 ECHO SERVER ISHGA TUSHDI")
    print("=" * 45)
    print(f"  Host : {HOST}")
    print(f"  Port : {PORT}")
    print(f"  Vaqt : {vaqt()}")
    print("=" * 45)
    print("  Klientlarni kutmoqda... (Ctrl+C = stop)")
    print("=" * 45)

    try:
        while True:
            # Yangi klient ulanishini kutish (blokirovchi)
            klient_sock, manzil = server_sock.accept()

            # Har klient uchun alohida thread
            t = threading.Thread(
                target=klient_handler,
                args=(klient_sock, manzil),
                daemon=True   # Asosiy dastur tugasa thread ham tugaydi
            )
            t.start()

            # Hozir nechta klient bor
            faol = threading.active_count() - 1
            print(f"[{vaqt()}] 📊 Faol ulanishlar: {faol}")

    except KeyboardInterrupt:
        print(f"\n[{vaqt()}] 🔴 Server to'xtatildi (Ctrl+C)")
    finally:
        server_sock.close()
        print(f"[{vaqt()}] 🔒 Server socket yopildi")


if __name__ == "__main__":
    serverni_ishga_tushir()
```

---

klient.py

```
# klient.py
import socket
import sys


HOST = "127.0.0.1"   # Localhost (o'z kompyuter)
PORT = 12345
```

```python
def serverga_ulan():
    # Socket yaratish
    klient_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    klient_sock.settimeout(10)   # 10 soniya timeout

    try:
        print("=" * 45)
        print("        📡 ECHO KLIENT")
        print("=" * 45)
        print(f"  Serverga ulanilmoqda: {HOST}:{PORT}")

        # Serverga ulanish
        klient_sock.connect((HOST, PORT))
        klient_sock.settimeout(None)   # Ulangandan keyin timeout o'chirish

        print("  ✅ Ulanish muvaffaqiyatli!")
        print("=" * 45)
        print("  💡 Buyruqlar:")
        print("     'exit' → Chiqish")
        print("     'ping' → Server tirik ekanini tekshirish")
        print("     Boshqa → Katta harfga o'giriladi")
        print("=" * 45)

        while True:
            # Foydalanuvchidan xabar olish
            try:
                xabar = input("\n📝 Xabar: ").strip()
            except (KeyboardInterrupt, EOFError):
                print("\n👋 Chiqilmoqda...")
                xabar = "exit"

            # Bo'sh xabar
            if not xabar:
                print("⚠️  Bo'sh xabar yuborib bo'lmaydi!")
                continue

            # Xabar yuborish
            klient_sock.sendall(xabar.encode("utf-8"))

            # Javob olish
            javob = klient_sock.recv(1024).decode("utf-8")
            print(f"🔊 Server javobi: '{javob}'")

            # Exit buyrug'i
            if xabar.lower() == "exit":
                break

    except ConnectionRefusedError:
        print(f"❌ Xato: Server {HOST}:{PORT} da ishlamayapti!")
        print("   Avval server.py ni ishga tushiring.")
        sys.exit(1)
    except socket.timeout:
        print("⏱️  Xato: Server 10 soniya ichida javob bermadi!")
        sys.exit(1)
    except Exception as e:
```

```
        print(f"❌ Kutilmagan xato: {e}")
    finally:
        klient_sock.close()
        print("🔒 Klient socket yopildi. Xayr!")


if __name__ == "__main__":
    serverga_ulan()
```

Ishga Tushirish

```
# Terminal 1 — Avval server:
python server.py

# Terminal 2 — Keyin klient:
python klient.py

# Bir vaqtda ko'p klient (Terminal 3, 4...):
python klient.py
python klient.py
```

Ishga Tushirish Natijasi

```
# server.py chiqishi:
══════════════════════════════════════════════════
       🚀 ECHO SERVER ISHGA TUSHDI
══════════════════════════════════════════════════
  Host : 0.0.0.0
  Port : 12345
  Vaqt : 14:23:01
══════════════════════════════════════════════════
  Klientlarni kutmoqda... (Ctrl+C = stop)
══════════════════════════════════════════════════
[14:23:05] ✅ Yangi ulanish: 127.0.0.1:54321
[14:23:05] 📊 Faol ulanishlar: 1
[14:23:08] 📨 Keldi [127.0.0.1]: 'salom dunyo'
[14:23:08] 📤 Yuborildi [127.0.0.1]: 'SALOM DUNYO'
[14:23:11] 📨 Keldi [127.0.0.1]: 'ping'
[14:23:11] 📤 Yuborildi [127.0.0.1]: 'PONG'
[14:23:14] 📨 Keldi [127.0.0.1]: 'exit'
[14:23:14] 👋 Klient chiqdi: 127.0.0.1
[14:23:14] 🔒 Socket yopildi: 127.0.0.1

# klient.py chiqishi:
══════════════════════════════════════════════════
       📡 ECHO KLIENT
══════════════════════════════════════════════════
  Serverga ulanilmoqda: 127.0.0.1:12345
  ✅ Ulanish muvaffaqiyatli!
══════════════════════════════════════════════════
  💡 Buyruqlar:
     'exit' → Chiqish
     'ping' → Server tirik ekanini tekshirish
```

```
    Boshqa → Katta harfga o'giriladi
═══════════════════════════════════════════
```

```
📝 Xabar: salom dunyo
📢 Server javobi: 'SALOM DUNYO'

📝 Xabar: ping
📢 Server javobi: 'PONG'

📝 Xabar: exit
📢 Server javobi: 'Xayr! Ulanish yopildi.'
🔒 Klient socket yopildi. Xayr!
```

Dastur Arxitekturasi

```
server.py:
  serverni_ishga_tushir()
    ├── socket() → AF_INET, SOCK_STREAM
    ├── setsockopt() → SO_REUSEADDR
    ├── bind() → HOST:PORT
    ├── listen(5) → Navbat
    └── accept() → Yangi klient
          ↓
    threading.Thread(klient_handler)
          ↓
    klient_handler():
      ├── recv(1024) → Xabar qabul
      ├── .upper()   → Katta harf
      └── send()     → Javob yuborish

klient.py:
  serverga_ulan()
    ├── socket() → AF_INET, SOCK_STREAM
    ├── settimeout(10) → Ulanish vaqti
    ├── connect() → HOST:PORT
    └── Loop:
          ├── input() → Foydalanuvchi xabari
          ├── sendall() → Yuborish
          └── recv()    → Javob olish
```

Asosiy Nazariy Tushunchalar

```
SO_REUSEADDR:
  Server yopilgandan keyin port 2-4 daqiqa band turadi
  (TIME_WAIT holati). setsockopt bilan darhol qayta ishlatish mumkin.

daemon=True (Thread):
  Asosiy dastur (Ctrl+C) to'xtatilsa →
  Barcha daemon threadlar ham avtomatik tugaydi.
  False bo'lsa → Klientlar bilan ishlab turgan
  threadlar dastur tugashini to'sib qoladi.
```

```
settimeout(10):
  Klient server bilan ulanishga 10 soniya urinadi.
  Javob kelmasa → socket.timeout xatosi.
  Ulangandan keyin → settimeout(None) → Cheksiz kutish.

.upper():
  "salom" → "SALOM"
  O'zbek harflari: "salom dunyo" → "SALOM DUNYO"
```

> 💡 **Eslab qol:** Echo server — socket dasturlashning **"Hello World"** i. Har qanday murakkab chat, fayl uzatish, yoki real-time dastur shu asosda quriladi. `threading` + `socket` kombinatsiyasi ko'p klientli serverning eng sodda va tushunarli arxitekturasi!

🔴 **SAVOL 48:**

`[PYQT5]` FOYDALANUVCHI ISMINI KIRITGANDA "XUSH KELIBSIZ, [ISM]!" DEB `QMESSAGEBOX` DIALOG OYNASI KO'RSATADIGAN DASTUR YOZING. MAYDON BO'SH QOLDIRILSA OGOHLANTIRISH BERING.

```python
import sys
from PyQt5.QtWidgets import (
    QApplication, QMainWindow, QWidget,
    QVBoxLayout, QHBoxLayout,
    QLabel, QLineEdit, QPushButton, QMessageBox
)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont


class XushKelibsiz(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Xush Kelibsiz Dasturi")
        self.setFixedSize(420, 320)
        self.setStyleSheet("background-color: #f0f4f8;")
        self.interfeys_yaratish()

    def interfeys_yaratish(self):
        markaziy = QWidget()
        self.setCentralWidget(markaziy)
        tartib = QVBoxLayout(markaziy)
        tartib.setContentsMargins(35, 30, 35, 30)
        tartib.setSpacing(16)

        # ── Sarlavha ──────────────────────────────
        sarlavha = QLabel("👋 Salom!")
        sarlavha.setAlignment(Qt.AlignCenter)
        sarlavha.setStyleSheet("""
            font-size: 28px;
            font-weight: bold;
            color: white;
            background-color: #3498db;
            border-radius: 12px;
```

```
    padding: 14px;
""")
tartib.addWidget(sarlavha)

# ── Izoh matni ──────────────────────────────
izoh = QLabel("Ismingizni kiriting va 'Kirish' tugmasini bosing.")
izoh.setAlignment(Qt.AlignCenter)
izoh.setWordWrap(True)
izoh.setStyleSheet("""
    font-size: 13px;
    color: #7f8c8d;
    padding: 4px;
""")
tartib.addWidget(izoh)

# ── Ism label ───────────────────────────────
ism_label = QLabel("Ismingiz:")
ism_label.setStyleSheet("""
    font-size: 14px;
    font-weight: bold;
    color: #2c3e50;
""")
tartib.addWidget(ism_label)

# ── Input maydon ────────────────────────────
self.ism_input = QLineEdit()
self.ism_input.setPlaceholderText("Masalan: Alibek...")
self.ism_input.setMinimumHeight(45)
self.ism_input.setStyleSheet("""
    QLineEdit {
        font-size: 15px;
        padding: 8px 14px;
        border: 2px solid #bdc3c7;
        border-radius: 10px;
        background-color: white;
        color: #2c3e50;
    }
    QLineEdit:focus {
        border: 2px solid #3498db;
        background-color: #eaf4fb;
    }
""")
# Enter bosilganda ham ishlashi uchun:
self.ism_input.returnPressed.connect(self.salomlash)
tartib.addWidget(self.ism_input)

# ── Xato xabari (yashirin) ──────────────────
self.xato_label = QLabel("")
self.xato_label.setAlignment(Qt.AlignCenter)
self.xato_label.setStyleSheet("""
    font-size: 12px;
    color: #e74c3c;
    font-style: italic;
""")
tartib.addWidget(self.xato_label)

# ── Tugmalar ────────────────────────────────
```

```
        tugmalar_tartib = QHBoxLayout()
        tugmalar_tartib.setSpacing(12)

        # Kirish tugmasi:
        kirish_tugma = QPushButton("✅  Kirish")
        kirish_tugma.setMinimumHeight(46)
        kirish_tugma.setStyleSheet("""
            QPushButton {
                background-color: #27ae60;
                color: white;
                font-size: 15px;
                font-weight: bold;
                border-radius: 10px;
                border: none;
            }
            QPushButton:hover   { background-color: #2ecc71; }
            QPushButton:pressed { background-color: #1e8449;
                                  padding-top: 2px; }
        """)
        kirish_tugma.clicked.connect(self.salomlash)
        tugmalar_tartib.addWidget(kirish_tugma)

        # Tozalash tugmasi:
        tozala_tugma = QPushButton("🗑  Tozalash")
        tozala_tugma.setMinimumHeight(46)
        tozala_tugma.setStyleSheet("""
            QPushButton {
                background-color: #e74c3c;
                color: white;
                font-size: 15px;
                font-weight: bold;
                border-radius: 10px;
                border: none;
            }
            QPushButton:hover   { background-color: #ff6b6b; }
            QPushButton:pressed { background-color: #c0392b;
                                  padding-top: 2px; }
        """)
        tozala_tugma.clicked.connect(self.tozalash)
        tugmalar_tartib.addWidget(tozala_tugma)

        tartib.addLayout(tugmalar_tartib)

        # Fokusni inputga o'rnatish:
        self.ism_input.setFocus()

    # ── Asosiy funksiyalar ──────────────────────────

    def salomlash(self):
        ism = self.ism_input.text().strip()

        # Bo'sh maydon tekshiruvi:
        if not ism:
            # Input chegarasini qizilga o'zgartirish:
            self.ism_input.setStyleSheet("""
                QLineEdit {
                    font-size: 15px;
```

```
                padding: 8px 14px;
                border: 2px solid #e74c3c;
                border-radius: 10px;
                background-color: #fdf2f2;
                color: #2c3e50;
            }
            QLineEdit:focus {
                border: 2px solid #e74c3c;
                background-color: #fdf2f2;
            }
        """)
        self.xato_label.setText("⚠️  Iltimos, ismingizni kiriting!")

        # Ogohlantirish dialogi:
        ogohlantirish = QMessageBox(self)
        ogohlantirish.setWindowTitle("Diqqat!")
        ogohlantirish.setIcon(QMessageBox.Warning)
        ogohlantirish.setText("Ism maydoni bo'sh qoldirildi!")
        ogohlantirish.setInformativeText(
            "Iltimos, ismingizni kiriting va qayta urinib ko'ring."
        )
        ogohlantirish.setStandardButtons(QMessageBox.Ok)
        ogohlantirish.setStyleSheet("""
            QMessageBox {
                background-color: #fdf2f2;
            }
            QLabel {
                color: #2c3e50;
                font-size: 13px;
                min-width: 260px;
            }
            QPushButton {
                background-color: #e74c3c;
                color: white;
                border-radius: 6px;
                padding: 6px 20px;
                font-weight: bold;
                min-width: 70px;
            }
            QPushButton:hover { background-color: #ff6b6b; }
        """)
        ogohlantirish.exec_()
        self.ism_input.setFocus()
        return

    # Muvaffaqiyatli salomlash:
    self.xato_label.setText("")
    self.ism_input.setStyleSheet("""
        QLineEdit {
            font-size: 15px;
            padding: 8px 14px;
            border: 2px solid #27ae60;
            border-radius: 10px;
            background-color: #f0fff4;
            color: #2c3e50;
        }
    """)
```

```
        salom = QMessageBox(self)
        salom.setWindowTitle("Xush Kelibsiz!")
        salom.setIcon(QMessageBox.Information)
        salom.setText(f"Xush kelibsiz, {ism}! 🎉")
        salom.setInformativeText(
            f"Siz dasturga muvaffaqiyatli kirdingiz.\n"
            f"Sizning ismingiz: {ism}"
        )
        salom.setStandardButtons(QMessageBox.Ok)
        salom.setStyleSheet("""
            QMessageBox {
                background-color: #f0fff4;
            }
            QLabel {
                color: #2c3e50;
                font-size: 14px;
                min-width: 260px;
            }
            QPushButton {
                background-color: #27ae60;
                color: white;
                border-radius: 6px;
                padding: 6px 20px;
                font-weight: bold;
                min-width: 70px;
            }
            QPushButton:hover { background-color: #2ecc71; }
        """)
        salom.exec_()

    def tozalash(self):
        self.ism_input.clear()
        self.xato_label.setText("")
        self.ism_input.setStyleSheet("""
            QLineEdit {
                font-size: 15px;
                padding: 8px 14px;
                border: 2px solid #bdc3c7;
                border-radius: 10px;
                background-color: white;
                color: #2c3e50;
            }
            QLineEdit:focus {
                border: 2px solid #3498db;
                background-color: #eaf4fb;
            }
        """)
        self.ism_input.setFocus()


if __name__ == "__main__":
    app = QApplication(sys.argv)
    oyna = XushKelibsiz()
    oyna.show()
    sys.exit(app.exec_())
```

Dastur Oqimi

```
Foydalanuvchi ism yozadi
        ↓
"Kirish" tugmasi yoki Enter
        ↓
    ism bo'shmi?
    ↙ Ha          ↘ Yo'q
Input qizil     Input yashil
Xato label      QMessageBox.Information
QMessageBox     "Xush kelibsiz, [ism]!"
.Warning
```

Asosiy Tushunchalar

```
returnPressed signal:
  Enter bosilganda tugma bosilgandek ishlaydi
  self.ism_input.returnPressed.connect(self.salomlash)

.strip():
  "  Alibek  " → "Alibek"
  Faqat bo'shliqdan iborat " " → "" (bo'sh)
  → Bo'sh maydon tekshiruvi to'g'ri ishlaydi

Dinamik StyleSheet:
  Xato → Qizil chegara + qizil fon
  OK   → Yashil chegara + yashil fon
  Reset→ Kulrang chegara + oq fon

QMessageBox.Warning    → ⚠️  Sariq uchburchak ikona
QMessageBox.Information→ ℹ️  Ko'k "i" ikona
```

> 💡 **Eslab qol:** `.strip()` — foydalanuvchi kiritishini tozalashning **birinchi qadami**. Faqat bo'shliqdan iborat `"   "` matn texnik jihatdan bo'sh emas, lekin mantiqan bo'sh. `.strip()` bu muammoni hal qiladi!

🔴 **SAVOL 49:**

`[SOCKET]` KLIENT SERVERGA O'Z NOMINI YUBORSIN, SERVER BARCHA ULANGAN KLIENTLAR RO'YXATINI QAYTARSIN. `THREADING` YORDAMIDA BIR NECHTA KLIENTNI BIR VAQTDA QABUL QILUVCHI SERVER YOZING.

---

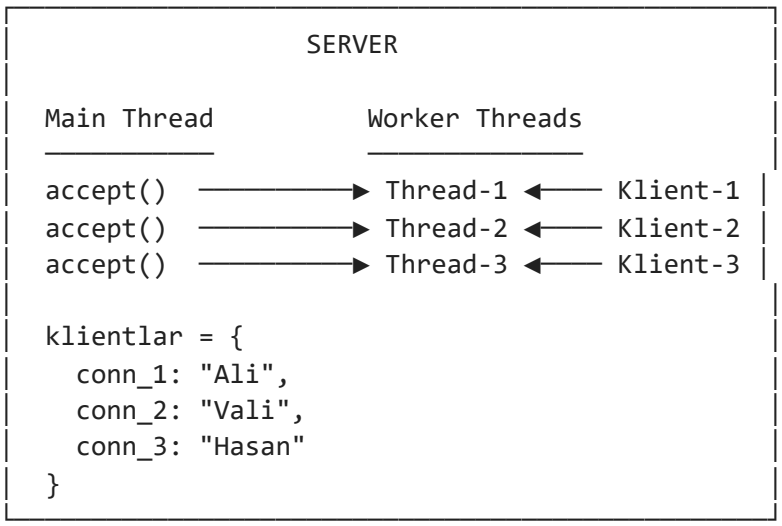
1. Arxitektura — Qanday Ishlaydi?

```
UMUMIY TUZILMA:

  SERVER (bir dona)                KLIENTLAR (ko'p)
  ─────────────────                ────────────────
  1. Port da tinglaydi             1. Serverga ulanadi
```

```
2. Har klient → alohida thread      2. O'z nomini yuboradi
3. Nomni ro'yxatga qo'shadi         3. Barcha ro'yxatni oladi
4. Ro'yxatni qaytaradi              4. Chiqaradi
```

Vizual ko'rinish:




```
MUAMMO: Bir nechta thread bir vaqtda ro'yxatni o'qisa/yozsa
        → Race Condition (ma'lumot buzilishi)
YECHIM: threading.Lock() — faqat bitta thread o'zgartirsin
```

Threading zarurati:

```
THREADSIZ (noto'g'ri):

  Server accept() → Klient-1 bilan ishlaydi...
  Bu vaqtda Klient-2 ulanmoqchi → KUTADI ♻️
  Klient-1 tugaganda Klient-2 qabul qilinadi
  → Ketma-ket, parallel emas

THREADING BILAN (to'g'ri):

  Server accept() → Thread-1 (Klient-1 bilan) ─┐
  Server accept() → Thread-2 (Klient-2 bilan) ─┤ Parallel!
  Server accept() → Thread-3 (Klient-3 bilan) ─┘
  → Bir vaqtda, parallel
```

---

2. Server Kodi — To'liq Izohlar Bilan

```python
# server.py

import socket      # Tarmoq ulanishlari uchun
import threading   # Parallel threadlar uchun

# ═══════════════════════════════════════════════════
# GLOBAL O'ZGARUVCHILAR
# ═══════════════════════════════════════════════════

# Ulangan klientlar ro'yxati: {connection: "ism"} ko'rinishida
```

```python
# dictionary ishlatiladi — connection ob'ekti kalit bo'ladi
klientlar = {}

# Lock — bir vaqtda faqat bitta thread klientlar ni o'zgartirsin
# Lock olmasa: Thread-1 yozayotganda Thread-2 ham yozishi mumkin
# Bu "race condition" deb ataladi va ma'lumotni buzadi
lock = threading.Lock()


# ═══════════════════════════════════════════════════
# KLIENT BILAN ISHLASH FUNKSIYASI
# Har bir klient uchun alohida thread shu funksiyani ishlatadi
# ═══════════════════════════════════════════════════

def klient_bilan_ishla(conn, addr):
    """
    conn  — klient bilan bog'langan socket ob'ekti
    addr  — klientning (IP, port) manzili
    Bu funksiya har bir klient uchun alohida threadda ishlaydi
    """

    print(f"[+] Yangi ulanish: {addr[0]}:{addr[1]}")

    try:
        # ── QADAM 1: Klientdan ismni qabul qilish ──────────
        # recv(1024) → maksimum 1024 bayt qabul qil
        # .decode('utf-8') → baytlarni stringga aylantir
        # .strip() → bosh/oxirdagi bo'sh joylarni o'chir
        xabar = conn.recv(1024).decode('utf-8').strip()

        # Agar klient hech narsa yubormasa yoki
        # ulanish uzilsa — xabar bo'sh bo'ladi
        if not xabar:
            print(f"[-] {addr} bo'sh xabar yubordi, ulanish uzildi")
            return   # Funksiyadan chiqish → thread tugaydi

        # Xabar formatini tekshirish: "ISM:Ali" ko'rinishida
        # Bu ixtiyoriy format — muloqot protokoli (protocol)
        if xabar.startswith("ISM:"):
            ism = xabar[4:].strip()   # "ISM:" prefiksini olib tashlash
        else:
            ism = xabar               # Prefiks yo'q → to'g'ridan ism

        print(f"[*] {addr[0]}:{addr[1]} → ism: '{ism}'")

        # ── QADAM 2: Ismni global ro'yxatga qo'shish ────────
        # with lock: → Lock ni qo'llash (olish)
        # Bu blok tugaganda lock avtomatik ozod qilinadi
        # Boshqa thread shu vaqtda bu kodni ishlatishni KUTADI
        with lock:
            klientlar[conn] = ism
            # Hozirgi paytdagi ro'yxat nusxasini olish
            # Bu nusxa lock bo'lganida olingan → to'liq ro'yxat
            hozirgi_royxat = list(klientlar.values())

        print(f"[*] Hozirgi ro'yxat: {hozirgi_royxat}")
```

```
            # ── QADAM 3: Barcha ismlar ro'yxatini klientga yuborish
            # Ismlarni vergul bilan birlashtirish
            # ["Ali", "Vali", "Hasan"] → "Ali, Vali, Hasan"
            royxat_matni = ", ".join(hozirgi_royxat)

            # Javob xabarini formatlash
            javob = f"ROYXAT:{royxat_matni}"

            # .encode('utf-8') → stringni baytlarga aylantir
            # Socket faqat bayt (bytes) yuboradi, string emas!
            conn.send(javob.encode('utf-8'))

            print(f"[>] {addr} ga ro'yxat yuborildi: {royxat_matni}")

        except ConnectionResetError:
            # Klient dasturi to'satdan yopilsa shu xato chiqadi
            print(f"[!] {addr} ulanishni to'satdan uzdi")

        except OSError as e:
            # Socket bilan boshqa muammolar
            print(f"[!] {addr} socket xato: {e}")

        finally:
            # ── QADAM 4: Klientni ro'yxatdan chiqarish ──────────
            # finally → xato bo'lsa ham, bo'lmasa ham ishga tushadi
            # Bu kafolat: connection DOIM yopiladi va ro'yxatdan chiqariladi
            with lock:
                if conn in klientlar:
                    chiqib_ketgan = klientlar.pop(conn)
                    print(f"[-] '{chiqib_ketgan}' ro'yxatdan chiqdi")

            # Socket ulanishini yopish — resurs bo'shatish
            conn.close()
            print(f"[-] {addr} ulanishi yopildi")


# ═══════════════════════════════════════════════════════
# SERVER ASOSIY FUNKSIYASI
# ═══════════════════════════════════════════════════════

def server_ishga_tushir(host='127.0.0.1', port=55000):
    """
    host — server qaysi IP da tinglaydi
           '127.0.0.1' → faqat shu kompyuterdan
           '0.0.0.0'   → tarmoqdagi barcha ulanishlardan
    port — server qaysi port da tinglaydi (0-65535)
           0-1023 → system portlar (root kerak)
           1024-65535 → oddiy foydalanuvchi portlari
    """

    # ── Socket yaratish ──────────────────────────────
    # socket.AF_INET   → IPv4 manzillar (192.168.x.x, 127.0.0.1)
    # socket.SOCK_STREAM → TCP protokoli (ishonchli, tartibli)
    # 'with' ishlatilsa → blok tugaganda socket avtomatik yopiladi
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_sock:

        # SO_REUSEADDR → Server yopilgandan darhol shu portni
```

```python
# qayta ishlatishga ruxsat beradi
# Onsiz: "Address already in use" xatosi ~60 soniya davom etadi
server_sock.setsockopt(
    socket.SOL_SOCKET,
    socket.SO_REUSEADDR,
    1
)

# ── Portni bog'lash (bind) ──────────────────────
# Server qaysi IP va portda "tinglash" ni bildiradi
server_sock.bind((host, port))

# ── Tinglashni boshlash (listen) ────────────────
# 5 → "backlog" — ulanishni kutayotganlar navbati
# Agar 5 ta klient bir vaqtda ulansa va accept() ularni
# ulgura olmasa, 5 tasi navbatda kutadi, qolganlar rad etiladi
server_sock.listen(5)

print(f"[SERVER] {host}:{port} da ishga tushdi")
print(f"[SERVER] Klientlarni kutmoqda... (Ctrl+C → to'xtatish)")
print("─" * 50)

# ── Asosiy tsikl: cheksiz klientlarni qabul qilish ─
while True:
    try:
        # accept() → Yangi klient ulanganda qaytadi
        # Bu BLOKLAYDI — klient kelgunga qadar kutadi
        # conn → yangi klient bilan socket
        # addr → (IP, port) tuple
        conn, addr = server_sock.accept()

        # ── Yangi thread yaratish ──────────────
        # target → thread ishlatadigan funksiya
        # args   → funksiyaga beriladigan argumentlar (tuple!)
        # daemon=True → asosiy dastur yopilsa, thread ham yopiladi
        #               Onsiz thread "zombi" bo'lib qolishi mumkin
        thread = threading.Thread(
            target=klient_bilan_ishla,
            args=(conn, addr),
            daemon=True
        )

        # Thread ni ishga tushirish
        # start() → thread parallel ishlashni boshlaydi
        # Bu yerda funksiya tugashini KUTMAYMIZ — parallel ketadi
        thread.start()

        # Hozir nechta aktiv thread borligini ko'rsatish
        # threading.active_count() → barcha threadlar (main + worker)
        # -1 → main thread o'zi hisoblanmasin
        aktiv = threading.active_count() - 1
        print(f"[SERVER] Aktiv klientlar: {aktiv} ta")

    except KeyboardInterrupt:
        # Ctrl+C bosilganda chiroyli to'xtatish
        print("\n[SERVER] To'xtatilmoqda...")
        break
```

```
        print("[SERVER] Server yopildi")


# ═══════════════════════════════════════════════════════
# DASTURNI ISHGA TUSHIRISH
# ═══════════════════════════════════════════════════════

if __name__ == '__main__':
    # Bu shart: faqat to'g'ridan to'g'ri ishga tushirilganda
    # boshqa fayldan import qilinsa — ishga tushmaydi
    server_ishga_tushir()
```

---

3. Klient Kodi — To'liq Izohlar Bilan

```
# klient.py

import socket   # Tarmoq ulanishi uchun

def serverga_ulan(host='127.0.0.1', port=55000):
    """
    host → server manzili
    port → server porti (server bilan bir xil bo'lishi shart!)
    """

    # ── Ismni foydalanuvchidan olish ──────────────────
    ism = input("Ismingizni kiriting: ").strip()

    # Bo'sh ism tekshiruvi
    if not ism:
        print("[!] Ism bo'sh bo'lishi mumkin emas!")
        return

    print(f"[*] {host}:{port} ga ulanilmoqda...")

    try:
        # ── Socket yaratish va ulaning ────────────────
        # 'with' → blok tugaganda socket avtomatik yopiladi
        with socket.socket(
                socket.AF_INET,
                socket.SOCK_STREAM) as sock:

            # connect() → serverga ulanish
            # Bu sinxron: ulanish o'rnatilgunga qadar kutadi
            # Server ishlamasa → ConnectionRefusedError
            sock.connect((host, port))
            print(f"[+] Serverga ulandi!")

            # ── Ismni yuborish ────────────────────────
            # "ISM:" prefiksi qo'shiladi → server bu formatni kutadi
            # encode('utf-8') → string → bytes (socket bayt yuboradi)
            xabar = f"ISM:{ism}"
            sock.send(xabar.encode('utf-8'))
            print(f"[>] Yuborildi: '{xabar}'")
```

```
            # ── Javobni qabul qilish ──────────────────────
            # recv(4096) → maksimum 4096 bayt qabul qil
            # Ko'p klient bo'lsa ro'yxat uzun bo'lishi mumkin
            # shuning uchun 1024 emas 4096 tanlandı
            javob = sock.recv(4096).decode('utf-8').strip()

            if not javob:
                print("[!] Serverdan bo'sh javob keldi")
                return

            # ── Javobni tahlil qilish ─────────────────────
            # Kutilgan format: "ROYXAT:Ali, Vali, Hasan"
            if javob.startswith("ROYXAT:"):
                royxat_qismi = javob[7:]   # "ROYXAT:" ni olib tashlash

                # Vergul bo'yicha ajratish va tozalash
                ismlar = [i.strip()
                          for i in royxat_qismi.split(",")
                          if i.strip()]

                print("\n" + "=" * 40)
                print("  HOZIR ULANGAN KLIENTLAR:")
                print("=" * 40)

                for tartib, klient_ismi in enumerate(ismlar, start=1):
                    # O'zini belgilash uchun "(siz)" qo'shish
                    belgi = " ← siz" if klient_ismi == ism else ""
                    print(f"  {tartib}. {klient_ismi}{belgi}")

                print("=" * 40)
                print(f"  Jami: {len(ismlar)} ta klient")
                print("=" * 40)
            else:
                # Kutilmagan format
                print(f"[?] Noma'lum javob: {javob}")

    except ConnectionRefusedError:
        # Server ishlamayapti yoki port noto'g'ri
        print(f"[!] Ulanib bo'lmadi: {host}:{port}")
        print("[!] Server ishga tushganmi? Port to'g'rimi?")

    except TimeoutError:
        # Ulanish vaqti tugadi
        print(f"[!] Ulanish vaqti tugadi: {host}:{port}")

    except OSError as e:
        print(f"[!] Socket xato: {e}")


# ═══════════════════════════════════════════════════════
# DASTURNI ISHGA TUSHIRISH
# ═══════════════════════════════════════════════════════

if __name__ == '__main__':
    serverga_ulan()
```

---

4. Ishlash Natijasi — Terminal Ko'rinishi

```
──────────────────────────────────────────────────────────────
SERVER terminali:

  [SERVER] 127.0.0.1:55000 da ishga tushdi
  [SERVER] Klientlarni kutmoqda... (Ctrl+C → to'xtatish)
  ──────────────────────────────────────────────────────────
  [+] Yangi ulanish: 127.0.0.1:52341
  [SERVER] Aktiv klientlar: 1 ta
  [*] 127.0.0.1:52341 → ism: 'Ali'
  [*] Hozirgi ro'yxat: ['Ali']
  [>] (127.0.0.1, 52341) ga ro'yxat yuborildi: Ali
  [-] 'Ali' ro'yxatdan chiqdi
  [-] (127.0.0.1, 52341) ulanishi yopildi

  [+] Yangi ulanish: 127.0.0.1:52342
  [+] Yangi ulanish: 127.0.0.1:52343
  [SERVER] Aktiv klientlar: 2 ta
  [SERVER] Aktiv klientlar: 3 ta
  [*] 127.0.0.1:52342 → ism: 'Vali'
  [*] 127.0.0.1:52343 → ism: 'Hasan'
  [*] Hozirgi ro'yxat: ['Vali', 'Hasan']
  [*] Hozirgi ro'yxat: ['Vali', 'Hasan']

──────────────────────────────────────────────────────────────
KLIENT-1 terminali (Ali):

  Ismingizni kiriting: Ali
  [*] 127.0.0.1:55000 ga ulanilmoqda...
  [+] Serverga ulandi!
  [>] Yuborildi: 'ISM:Ali'
  ═══════════════════════════════════════
    HOZIR ULANGAN KLIENTLAR:
  ═══════════════════════════════════════
    1. Ali ← siz
  ═══════════════════════════════════════
    Jami: 1 ta klient
  ═══════════════════════════════════════

──────────────────────────────────────────────────────────────
KLIENT-2 terminali (Vali) — bir vaqtda:

  Ismingizni kiriting: Vali
  [*] 127.0.0.1:55000 ga ulanilmoqda...
  [+] Serverga ulandi!
  [>] Yuborildi: 'ISM:Vali'
  ═══════════════════════════════════════
    HOZIR ULANGAN KLIENTLAR:
  ═══════════════════════════════════════
    1. Vali ← siz
    2. Hasan
  ═══════════════════════════════════════
    Jami: 2 ta klient
  ═══════════════════════════════════════
```

---

5. Race Condition — Lock Zarurati

```
LOCK OLMASA NIMA BO'LADI?

  Vaqt →→→→→→→→→→→→→→→→→→→→→→

  Thread-1: klientlar ni o'qiydi...    ["Ali"]
                                              ↕ Ikkalasi bir vaqtda!
  Thread-2: klientlar ga yozadi...     klientlar["conn2"] = "Vali"

  Natija: Thread-1 "Ali, Vali" o'rniga faqat "Ali" ko'rishi mumkin
          Yoki ma'lumot strukturasi BUZILISHI mumkin
          Python dict thread-safe emas!

LOCK BILAN:

  Thread-1: lock.acquire() ← Lock oldi
  Thread-2:                 ← KUTADI (lock band)
  Thread-1: klientlar ni o'qidi
  Thread-1: lock.release() ← Lock qo'ydi
  Thread-2: lock.acquire() ← Endi Thread-2 lock oldi
  Thread-2: klientlar ga yozdi
  Thread-2: lock.release() ← Tugadi

  'with lock:' → acquire() va release() avtomatik

MUAMMO: Lock "deadlock" ga olib kelishi mumkin
  Thread-1: Lock-A oldi, Lock-B kutmoqda
  Thread-2: Lock-B oldi, Lock-A kutmoqda
  → Ikkalasi cheksiz kutadi!

YECHIM: Bir dona lock ishlatish (bu misolda to'g'ri)
```

---

6. Protokol — Muloqot Formati

```
MULOQOT PROTOKOLI (qo'lda belgilangan):

  KLIENT → SERVER:
  ┌──────────────────────────────┐
  │  ISM:<ism_matni>             │
  │  Misol: "ISM:Ali Valiyev"    │
  └──────────────────────────────┘

  SERVER → KLIENT:
  ┌──────────────────────────────────────────────┐
  │  ROYXAT:<ism1>, <ism2>, <ism3>               │
  │  Misol: "ROYXAT:Ali Valiyev, Vali, Hasan"    │
  └──────────────────────────────────────────────┘

NIMA UCHUN PREFIX (ISM:, ROYXAT:)?
  Prefiks yo'q bo'lsa:
    "Ali" → Bu ism? Xato xabar? Boshqa ma'lumot?
  Prefiks bor:
    "ISM:Ali" → Aniq: bu ism yuborish
```

```
    "ROYXAT:Ali,Vali" → Aniq: bu ro'yxat javobi
  → Protokol muloqotni aniqlashtiradi
```

---

7. Xulosa

```
DASTUR TUZILMASI:

  SERVER:
    socket(AF_INET, SOCK_STREAM) → TCP socket
    setsockopt(SO_REUSEADDR)     → port darhol qayta ishlashi
    bind(host, port)             → IP va port bog'lash
    listen(5)                    → Tinglashni boshlash
    while True:
      accept()                   → Yangi klient kutish (blok)
      Thread(target=funksiya)    → Alohida thread yaratish
      thread.start()             → Parallel ishga tushirish
    ──────────────────────────────────────────────────
    klient_bilan_ishla():
      recv(1024)                 → Ismni qabul qilish
      with lock: dict.update()   → Xavfsiz ro'yxatga qo'shish
      send(ro'yxat)              → Javob yuborish
      finally: conn.close()      → Doim yopish

  KLIENT:
    socket(AF_INET, SOCK_STREAM) → TCP socket
    connect(host, port)          → Serverga ulanish
    send("ISM:ism")              → Ismni yuborish
    recv(4096)                   → Ro'yxatni qabul qilish
    (yopiladi — 'with' tufayli)

  THREADING + LOCK:
    Thread → Har klient parallel ishlaydi
    Lock   → klientlar dict ni xavfsiz o'zgartirish
    daemon=True → Asosiy dastur yopilsa thread ham tugaydi
```

> 💡 **Eslab qol:** `recv()` kafolat bermaydi — **bir chaqiruvda barcha baytlar kelmaydi**. Kichik ma'lumot (bu misol) uchun odatda muammo yo'q, lekin katta ma'lumot yuborilganda "framing" kerak: avval ma'lumot uzunligini, keyin ma'lumotning o'zini yuborish. Masalan: `"0012ISM:Ali Valiyev"` — dastlabki 4 raqam uzunlik, keyin matn. Bu **length-prefixed protocol** deb ataladi va real tizimlarda keng ishlatiladi.

🔴 **SAVOL 50:**

`[PYQT5 + SOCKET]` IP MANZIL VA PORT KIRITIB SERVERGA ULANA OLADIGAN, XABAR YUBORA OLADIGAN VA JAVOBNI OYNA ICHIDA KO‘RA OLADIGAN MINIMAL `GUI CHAT KLIENT` DASTUR YOZING.

---

1. Arxitektura — Qanday Ishlaydi?

```
UMUMIY TUZILMA:

  GUI KLIENT (bir oyna)
  ─────────────────────
  1. IP, Port, Ism kiritadi
  2. Serverga ulanadi
  3. Xabar yozib yuboradi
  4. Javobni oynada ko'radi

  Vizual ko'rinish:

  ┌──────────────────────────────────────────────┐
  │              GUI CHAT KLIENT                 │
  │                                              │
  │  [IP: 127.0.0.1] [Port: 55000] [Ism: Ali]    │
  │  [          Ulan          ]                  │
  │  ──────────────────────────────────────────  │
  │  [08:01] ✓ Serverga ulandi                   │
  │  [08:01] ➤ Ali: Salom!                       │
  │  [08:01] ◈ Server: ROYXAT: Ali, Vali         │
  │  ──────────────────────────────────────────  │
  │  [ Xabar yozing...              ] [Yuborish] │
  └──────────────────────────────────────────────┘

MUAMMO: sock.recv() bloklaydi → GUI muzlab qoladi
YECHIM: QThread — recv() ni alohida threadda ishlatish,
        signal orqali natijani GUI ga uzatish
```

QThread zarurati:

```
THREADSIZ (noto'g'ri):

  main thread → recv() ni kutadi...
  Bu vaqtda oyna → MUZLAYDI ♻️
  Foydalanuvchi hech narsa qila olmaydi
  → GUI javob bermay qoladi

QTHREAD BILAN (to'g'ri):

  main thread   → GUI ni boshqaradi (har doim jonli)
  QabulThread   → recv() ni kutadi (fon da)
  pyqtSignal    → xabar kelganda GUI ga xavfsiz uzatadi
  → Parallel, GUI hech qachon muzlamaydi
```

---

2. Klient GUI Kodi — To'liq Izohlar Bilan

```python
# klient_gui.py
# Ishga tushirish: pip install PyQt5 && python klient_gui.py

import sys
import socket
from datetime import datetime

from PyQt5.QtWidgets import (
```

```
    QApplication, QMainWindow, QWidget,
    QVBoxLayout, QHBoxLayout,
    QLabel, QLineEdit, QPushButton, QTextEdit, QFrame
)
from PyQt5.QtCore import QThread, pyqtSignal
from PyQt5.QtGui import QTextCursor


# ═══════════════════════════════════════════════════════
# XABAR QABUL QILUVCHI THREAD
# recv() bloklaydi → GUI muzlab qolmasin uchun QThread ishlatiladi
# ═══════════════════════════════════════════════════════

class QabulThread(QThread):
    """
    Serverdan xabarlarni fon threadida qabul qiladi.

    Qt qoidasi: GUI faqat main threaddan o'zgartiriladi.
    Boshqa threaddan to'g'ridan widget ga yozish → CRASH!
    Yechim: pyqtSignal → main thread ga xavfsiz uzatish.
    """

    # Signal — main thread ga xabar yuborish uchun
    # pyqtSignal(str) → string tip li signal
    yangi_xabar   = pyqtSignal(str)   # Yangi xabar keldi
    ulanish_uzildi = pyqtSignal(str)  # Ulanish uzildi (sabab bilan)

    def __init__(self, sock):
        super().__init__()
        self.sock = sock
        self._ishlayapti = True

    def run(self):
        """
        Thread ishga tushganda shu metod avtomatik chaqiriladi.
        Bu yerda cheksiz tsiklda serverdan xabar kutiladi.
        """
        while self._ishlayapti:
            try:
                # recv() → xabar kelgunga qadar BLOKLAYDI
                # Lekin bu QThread ichida → GUI muzlamaydi
                xabar = self.sock.recv(4096).decode('utf-8').strip()

                if not xabar:
                    # Bo'sh xabar → server ulanishni yopdi
                    self.ulanish_uzildi.emit("Server ulanishni yopdi")
                    break

                # Signal orqali main thread ga yuborish
                # emit() → signalni "otadi", main thread ushlab oladi
                self.yangi_xabar.emit(xabar)

            except OSError:
                # Socket yopildi (foydalanuvchi "Uzish" bosdi)
                if self._ishlayapti:
                    self.ulanish_uzildi.emit("Ulanish uzildi")
                break
```

```
    def to_xtatish(self):
        """Threadi xavfsiz to'xtatish."""
        self._ishlayapti = False


# ═══════════════════════════════════════════════════════════
# ASOSIY GUI OYNA
# ═══════════════════════════════════════════════════════════

class ChatKlient(QMainWindow):

    def __init__(self):
        super().__init__()

        # Holat o'zgaruvchilari
        self.sock         = None   # Aktiv socket
        self.qabul_thread = None   # Fon thread
        self.ulangan      = False  # Ulanish holati

        self._ui_qur()
        self.setWindowTitle("Socket Chat Klient")
        self.setMinimumSize(620, 480)

    # ─────────────────────────────────────────────────────
    # UI QURISH
    # ─────────────────────────────────────────────────────

    def _ui_qur(self):
        """Barcha widgetlarni yaratish va joylashtirish."""

        markaziy = QWidget()
        self.setCentralWidget(markaziy)
        layout = QVBoxLayout(markaziy)
        layout.setSpacing(8)
        layout.setContentsMargins(12, 12, 12, 12)

        # ── 1. Ulanish qatori ──────────────────────────
        ulanish_qator = QHBoxLayout()

        self.ip_input = QLineEdit("127.0.0.1")
        self.ip_input.setPlaceholderText("IP manzil")
        self.ip_input.setFixedWidth(130)

        self.port_input = QLineEdit("55000")
        self.port_input.setPlaceholderText("Port")
        self.port_input.setFixedWidth(75)

        self.ism_input = QLineEdit()
        self.ism_input.setPlaceholderText("Ismingiz")
        self.ism_input.setFixedWidth(110)

        self.ulanish_btn = QPushButton("Ulan")
        self.ulanish_btn.setFixedWidth(85)
        self.ulanish_btn.clicked.connect(self._ulanish_toggle)

        ulanish_qator.addWidget(QLabel("IP:"))
```

```
        ulanish_qator.addWidget(self.ip_input)
        ulanish_qator.addWidget(QLabel("Port:"))
        ulanish_qator.addWidget(self.port_input)
        ulanish_qator.addWidget(QLabel("Ism:"))
        ulanish_qator.addWidget(self.ism_input)
        ulanish_qator.addStretch()
        ulanish_qator.addWidget(self.ulanish_btn)

        # ── 2. Chat oynasi ──────────────────────────────
        # setReadOnly → foydalanuvchi to'g'ridan yozolmaydi
        self.chat_oyna = QTextEdit()
        self.chat_oyna.setReadOnly(True)

        # ── 3. Xabar yuborish qatori ────────────────────
        yuborish_qator = QHBoxLayout()

        self.xabar_input = QLineEdit()
        self.xabar_input.setPlaceholderText(
            "Xabar yozing... (Enter → yuborish)")
        # Enter bosilganda ham yuborsin
        self.xabar_input.returnPressed.connect(self._xabar_yuborish)
        self.xabar_input.setEnabled(False)   # Ulangunga qadar o'chiq

        self.yuborish_btn = QPushButton("Yuborish")
        self.yuborish_btn.setFixedWidth(90)
        self.yuborish_btn.clicked.connect(self._xabar_yuborish)
        self.yuborish_btn.setEnabled(False)  # Ulangunga qadar o'chiq

        yuborish_qator.addWidget(self.xabar_input)
        yuborish_qator.addWidget(self.yuborish_btn)

        # ── Layout yig'ish ──────────────────────────────
        layout.addLayout(ulanish_qator)
        layout.addWidget(self.chat_oyna, stretch=1)
        layout.addLayout(yuborish_qator)

    # ──────────────────────────────────────────────────────
    # ULANISH LOGIKASI
    # ──────────────────────────────────────────────────────

    def _ulanish_toggle(self):
        """Tugma: ulangan bo'lsa uzish, bo'lmasa ulanish."""
        if self.ulangan:
            self._uzish()
        else:
            self._ulanish()

    def _ulanish(self):
        """Serverga TCP ulanish."""

        host    = self.ip_input.text().strip()
        port_str = self.port_input.text().strip()
        ism     = self.ism_input.text().strip()

        # ── Validatsiya ─────────────────────────────────
        if not host:
            self._log("⚠ IP manzil kiritilmagan")
```

```
            return
        if not port_str.isdigit() or not (0 < int(port_str) < 65536):
            self._log("⚠ Port noto'g'ri (1–65535 bo'lishi kerak)")
            return
        if not ism:
            self._log("⚠ Ism kiritilmagan")
            return

        port = int(port_str)

        # ── Socket yaratish va ulanish ──────────────────────
        try:
            self.sock = socket.socket(
                socket.AF_INET, socket.SOCK_STREAM)

            # 5 soniya timeout — server javob bermasa qotib qolmaydi
            self.sock.settimeout(5)
            self.sock.connect((host, port))
            self.sock.settimeout(None)   # Ulanishdan keyin timeout olib tashlash

        except ConnectionRefusedError:
            self._log(f"✗ {host}:{port} — server javob bermadi")
            self.sock = None
            return
        except OSError as e:
            self._log(f"✗ Ulanish xatosi: {e}")
            self.sock = None
            return

        # ── Muvaffaqiyatli ulanish ──────────────────────────
        self.ulangan = True
        self._log(f"✓ {host}:{port} ga ulandi — salom, {ism}!")

        # UI ni ulangan holatga o'tkazish
        self.ulanish_btn.setText("Uzish")
        self.ip_input.setEnabled(False)
        self.port_input.setEnabled(False)
        self.ism_input.setEnabled(False)
        self.xabar_input.setEnabled(True)
        self.yuborish_btn.setEnabled(True)
        self.xabar_input.setFocus()

        # ── QabulThread ishga tushirish ─────────────────────
        # Signallarni ulash: thread signal otganda → shu metodlar chaqiriladi
        self.qabul_thread = QabulThread(self.sock)
        self.qabul_thread.yangi_xabar.connect(self._xabar_keldi)
        self.qabul_thread.ulanish_uzildi.connect(self._uzilish_hodisasi)
        self.qabul_thread.start()   # run() metodi boshlanadi

    def _uzish(self):
        """Serverdan ulanishni uzish."""

        # Avval threadi to'xtatish
        if self.qabul_thread:
            self.qabul_thread.to_xtatish()

        # Socketni yopish
```

```
        if self.sock:
            try:
                self.sock.shutdown(socket.SHUT_RDWR)
                self.sock.close()
            except OSError:
                pass
            self.sock = None

        self.ulangan = False
        self._log("◉ Ulanish uzildi")

        # UI ni uzilgan holatga qaytarish
        self.ulanish_btn.setText("Ulan")
        self.ip_input.setEnabled(True)
        self.port_input.setEnabled(True)
        self.ism_input.setEnabled(True)
        self.xabar_input.setEnabled(False)
        self.yuborish_btn.setEnabled(False)

    def _uzilish_hodisasi(self, sabab):
        """QabulThread dan kelgan uzilish signali."""
        if self.ulangan:
            self._uzish()
            self._log(f"✗ {sabab}")

    # ──────────────────────────────────────────────
    # XABAR YUBORISH
    # ──────────────────────────────────────────────

    def _xabar_yuborish(self):
        """Input maydonidagi xabarni serverga yuborish."""

        if not self.ulangan or not self.sock:
            self._log("⚠ Avval serverga ulaning")
            return

        matn = self.xabar_input.text().strip()
        if not matn:
            return

        try:
            # Socket faqat bytes yuboradi → encode() zarur
            self.sock.send(matn.encode('utf-8'))

            # O'z xabarimizni oynaga chiqarish
            ism = self.ism_input.text().strip()
            self._log(f"➤ {ism}: {matn}")
            self.xabar_input.clear()

        except OSError as e:
            self._log(f"✗ Yuborib bo'lmadi: {e}")
            self._uzish()

    # ──────────────────────────────────────────────
    # SERVERDAN XABAR KELDI (signal)
    # ──────────────────────────────────────────────
```

```
    def _xabar_keldi(self, xabar):
        """
        QabulThread yangi_xabar signalini otganda chaqiriladi.
        Bu main threadda ishlaydi → GUI xavfsiz yangilanadi.
        """
        # 49-savol protokoli bilan mos ishlash
        if xabar.startswith("ROYXAT:"):
            royxat = xabar[7:]
            self._log(f"◈ Server → Ulangan klientlar: {royxat}")
        else:
            self._log(f"◈ Server: {xabar}")

    # ──────────────────────────────────────────────────
    # YORDAMCHI: CHAT OYNASIGA YOZISH
    # ──────────────────────────────────────────────────

    def _log(self, matn):
        """Vaqt prefiksi bilan chat oynasiga qator qo'shish."""
        vaqt = datetime.now().strftime("%H:%M:%S")
        self.chat_oyna.moveCursor(QTextCursor.End)
        self.chat_oyna.insertPlainText(f"[{vaqt}] {matn}\n")
        self.chat_oyna.moveCursor(QTextCursor.End)

    # ──────────────────────────────────────────────────
    # OYNA YOPILGANDA
    # ──────────────────────────────────────────────────

    def closeEvent(self, event):
        """Oyna X tugmasi bilan yopilganda ulanishni xavfsiz uzish."""
        if self.ulangan:
            self._uzish()
        event.accept()


# ═══════════════════════════════════════════════════
# DASTURNI ISHGA TUSHIRISH
# ═══════════════════════════════════════════════════

if __name__ == '__main__':
    app = QApplication(sys.argv)
    oyna = ChatKlient()
    oyna.show()
    sys.exit(app.exec_())
```

---

3. Ishlash Natijasi — Terminal va Oyna Ko'rinishi

```
    ──────────────────────────────────────────────────

    SERVER terminali (49-savoldagi server):

      [SERVER] 127.0.0.1:55000 da ishga tushdi
      [+] Yangi ulanish: 127.0.0.1:54210
      [*] 127.0.0.1:54210 → ism: 'Ali'
      [>] (127.0.0.1, 54210) ga ro'yxat yuborildi: Ali
      [+] Yangi ulanish: 127.0.0.1:54211
      [*] 127.0.0.1:54211 → ism: 'Vali'
```

```
  [>] (127.0.0.1, 54211) ga ro'yxat yuborildi: Ali, Vali

──────────────────────────────────────────────────────────
GUI OYNA (Ali):

  ┌──────────────────────────────────────────────┐
  │ IP: [127.0.0.1] Port: [55000] Ism: [Ali] [Uzish]│
  │ ──────────────────────────────────────────── │
  │ [08:01:12] ✓ 127.0.0.1:55000 ga ulandi — Ali!  │
  │ [08:01:12] ◈ Server → Ulangan klientlar: Ali   │
  │ [08:01:45] ➤ Ali: Salom server!                │
  │ [08:01:45] ◈ Server: ROYXAT:Ali, Vali          │
  │ ──────────────────────────────────────────── │
  │ [ Xabar yozing...                ] [Yuborish] │
  └──────────────────────────────────────────────┘
```

---

4. QThread + Signal Mexanizmi

```
SIGNAL/SLOT MEXANIZMI:

  QabulThread (fon thread)        ChatKlient (main thread)
  ──────────────────────          ──────────────────────

  recv() → xabar keldi
  yangi_xabar.emit("ROYXAT:Ali") ──────────► _xabar_keldi()
                                              chat_oyna.insertPlainText()
                                              (GUI xavfsiz yangilandi)

  OSError → ulanish uzildi
  ulanish_uzildi.emit("sabab") ──────────► _uzilish_hodisasi()
                                             _uzish() chaqiriladi

SIGNAL ULASH:
  thread.yangi_xabar.connect(self._xabar_keldi)
  │                                  │
  │ Signal (thread da otiladi)     │ Slot (main thread da ishlaydi)
  └──────────────────────────────────┘
  Qt avtomatik thread xavfsizligini ta'minlaydi!

NIMA UCHUN EMIT XAVFSIZ?
  Qt "queued connection" mexanizmi:
  Thread emit() → Qt navbatiga qo'yadi
  Main thread   → navbatdan olib slotni chaqiradi
  → Ikki thread bir vaqtda GUI ga tegmaydi
```

---

5. Xulosa

```
DASTUR TUZILMASI:

  ChatKlient (QMainWindow):
    _ui_qur()            → Widgetlar, layoutlar yaratish
    _ulanish()           → socket.connect() + QabulThread start
    _uzish()             → thread.to_xtatish() + sock.close()
```

```
  _xabar_yuborish()     → sock.send(matn.encode())
  _xabar_keldi(xabar)   → chat_oyna ga yozish (signal slot)
  _log(matn)            → [HH:MM:SS] formatida qator qo'shish
  closeEvent()          → oyna yopilganda xavfsiz tozalash

QabulThread (QThread):
  run()                 → while True: sock.recv() → emit()
  to_xtatish()          → _ishlayapti = False

SIGNALLAR:
  yangi_xabar   → _xabar_keldi()
  ulanish_uzildi → _uzilish_hodisasi()

UI HOLATLARI:
  Uzilgan → IP/Port/Ism yoziladigan, xabar maydoni o'chiq
  Ulangan → IP/Port/Ism o'chiq, xabar maydoni yoziladigan
```

> 💡 **Eslab qol:** `QThread` ichida hech qachon `QLabel.setText()` yoki boshqa widget metodlarini chaqirma — bu **undefined behavior** va dastur istalgan payt **crash** bo'ladi. Faqat `pyqtSignal` orqali main thread ga ma'lumot uzat, widget ni shu yerda yangila. Bu Qt ning asosiy qoidasi: **"GUI is not thread-safe"**.